



Sorbonne Université

Rapport

Auto-assemblage distribué de robots Pogobot en grille orientée
afin d'afficher des motifs lumineux

LABIADH Yanis et GU David

18 mai 2026

Table des matières

1	Introduction	2
2	Objectif du travail réalisé	2
3	État de l'art	2
3.1	Contraintes des Pogobots	3
4	Présentation du robot	3
5	Organisation générale du programme	5
6	Description détaillée des modes	7
6.1	MODE_NORMAL	7
6.2	MODE_ROOT	8
6.3	MODE_ALIGN	8
6.4	MODE_CONNECTION_ROW et MODE_CONNECTION_COL	9
6.5	MODE_CHECK_CONNECTION_ROW et MODE_CHECK_CONNECTION_COL	10
6.6	MODE_ALIGN_TO_SEEK et MODE_SEEK_ROOT	10
6.7	MODE_CONNECTED	12
6.8	MODE_FLAG et MODE_FLAG_LIGHT	12
7	Difficultés rencontrées	13
8	Conclusion	14
A	Cahier des charges	16
A.1	Contexte du projet	16
A.2	Motivation	16
A.3	Pourquoi une grille et une orientation commune ?	16
A.4	Objectif général	17
A.5	Besoins exprimés par le « client »	17
A.6	Fonction principale attendue	17
A.7	Fonctions détaillées attendues	17
A.7.1	Gestion des cas locaux	17
A.7.2	Affichage du motif	18
A.8	Contraintes, périmètre et critères de réussite	18
B	Manuel d'utilisation	19
B.1	Matériel nécessaire	19
B.2	Compilation du programme	19
B.3	Configuration de l'identifiant du robot	19
B.4	Initialisation des robots	20
B.5	Déroulement d'une expérience	20
B.6	Couleurs utilisées	20
B.7	Conseils d'utilisation	20
B.8	Limites connues	21
B.9	Arrêt de l'expérience	21

1. Introduction

Dans le cadre de ce projet, il nous a été demandé de constituer un algorithme permettant à un système de robots de s'organiser de manière autonome afin de former une grille [7], dans un premier temps de taille 3×3 , puis d'utiliser cette structure pour afficher des motifs lumineux. Chaque robot joue alors le rôle d'une unité locale de l'affichage collectif, comparable à un pixel. Cette idée provient de travaux récents dans le domaine de la *swarm robotics* [6], où les robots assemblés en grille produisent collectivement un motif visuel à partir d'interactions locales [1, 2].

Le projet s'inscrit dans le domaine de la *swarm robotics*. Le cadre général, les objectifs et la logique du projet nous ont été expliqués par nos encadrants, Alessia Loi et Nicolas Bredeche.

2. Objectif du travail réalisé

L'objectif principal de notre travail a été de comprendre le fonctionnement du projet tel qu'il nous a été présenté, à l'aide du code non-fonctionnel d'une personne ayant déjà travaillé sur ce projet. Grâce à cela et divers autres documents, il nous a été demandé d'utiliser l'API des Pogobots afin de constituer notre propre algorithme.

Plus précisément, il nous a été donné ces différents objectifs :

- Reproduire les expériences de notre prédécesseur ;
- Concevoir un algorithme permettant à partir d'un robot racine déjà désigné et situé dans un socle de pouvoir constituer une grille ;
- Constituer une grille sans pré-sélection d'un robot racine, une racine doit pouvoir être désigné de manière autonome, après désignation il se connecte à un socle ;
- On supprime les socles de nos expériences, il faut pouvoir à n'importe quel endroit et pour n'importe quel robot racine désigné de manière autonome constituer une grille.

Enfin, il fallait pouvoir ajouter à notre algorithme les travaux expérimentaux sur la formation de motifs lumineux [1] ;

Il s'agissait également de retranscrire ces éléments dans un rapport clair, afin de disposer d'une trace écrite du travail réalisé et de l'avancement du projet.

3. État de l'art

La formation autonome de structures géométriques et l'auto-assemblage sont des problématiques fondamentales en *swarm robotics*. L'un des travaux les plus emblématiques dans ce domaine reste l'expérience menée par Rubenstein et al. [7], où un essaim de 1024 micro-robots (*Kilobots*) parvient à s'auto-assembler pour former des formes géométriques complexes en deux dimensions tel qu'une étoile ou bien la lettre 'K'.

Pour y parvenir, l'approche de Rubenstein repose sur la génération d'un système de coordonnées globales et virtuelles (x, y) partagé par l'essaim. À partir de robots « sources » (*seeds*) fixes, chaque robot calcule sa position continue en mesurant l'intensité des signaux infrarouges réfléchis par la surface de la table, la communication se fait aussi par réflexion des infrarouges. Une fois positionné dans ce repère global, le robot se déplace le long d'un champ de potentiel virtuel jusqu'à atteindre sa case cible sur un plan prédéfini.

Plus récemment, les recherches se sont orientées vers des méthodes n'utilisant plus un repère par système de coordonnées global au profit de structures purement locales. St-Onge et al. [8] ont proposé un algorithme de formation progressive de formes basé sur des graphes orientés acycliques. Dans ce modèle, les robots rejoignent la structure de manière autonome en suivant des règles d'alignement local par rapport à deux robots parents » déjà intégrés à la structure.

3.1. Contraintes des Pogobots

Bien que ces algorithmes soient performants, ils reposent tous sur une hypothèse matérielle cruciale : la capacité de chaque agent à estimer de manière continue la distance et l'orientation relative de plusieurs voisins simultanément (*range-and-bearing tracking*), ou à utiliser la réflexion globale d'un signal infrarouge.

La plateforme *Pogobot* [3] impose un schéma d'interaction fondamentalement différent à ceux de la littérature cité qui rend donc ces algorithmes inutilisables pour nos robots pour plusieurs raisons :

- **Communication discrète et directionnelle :** Contrairement aux Kilobots [7] qui émettent de manière omnidirectionnelle, le Pogobot communique via des émetteurs-récepteurs infrarouges localisés sur des faces spécifiques. La détection et la communication ne se font pas à distance continue, mais de manière discrète, presque exclusivement lorsque deux robots se font face ou s'alignent côte à côte.
- **Absence de mesure fine de distance :** Le matériel actuel ne permet pas d'obtenir un positionnement ou un calcul précis de coordonnées (x, y) par rapport à des voisins contrairement aux marXBot [8] [9] qui sont équipés d'un range-and-bearing sensor permettant de déterminer les positions des robots à ses voisins. De la même manière le Pogobot ne dispose pas de la capacité de mesurer l'intensité d'infrarouge comme le Kilobot pour obtenir une distance approximative à ses voisins.

Dès lors, il devient obligatoire de concevoir un algorithme adapté à ces contraintes strictes. Plutôt que de s'appuyer sur une géométrie continue, nous avons une méthode d'assemblage discrète. avec chacun des robots prenant une coordonnée (x, y) en utilisant l'information partagée par le robot sur lequel il se connecte, incrémentant de 1 une coordonnée précise. L'algorithme développé utilise une machine à états finis, forçant les robots à s'ancrer physiquement les un après les autres pour construire la grid case par case, une contrainte étant qu'une colonne ne peut pas être plus haute que la colonne d'origine, on construit de gauche à droite et de haut en bas.

4. Présentation du robot

Le projet repose sur l'utilisation de robots Pogobots [3], qui sont de petits robots mobiles, disposant de leur propre API, capables de se déplacer, de communiquer localement par infrarouge et d'afficher une information lumineuse. Dans notre cas, chaque robot constitue une unité locale du système collectif : il se déplace, détecte des voisins, échange des messages, puis peut s'intégrer à une structure plus grande.

Chaque robot possède plusieurs faces utilisées pour la communication et l'interprétation de son voisinage. Dans le cadre de notre projet, nous avons adopté la convention suivante :

- face 0 : avant ;
- face 1 : droite ;
- face 2 : arrière ;
- face 3 : gauche.

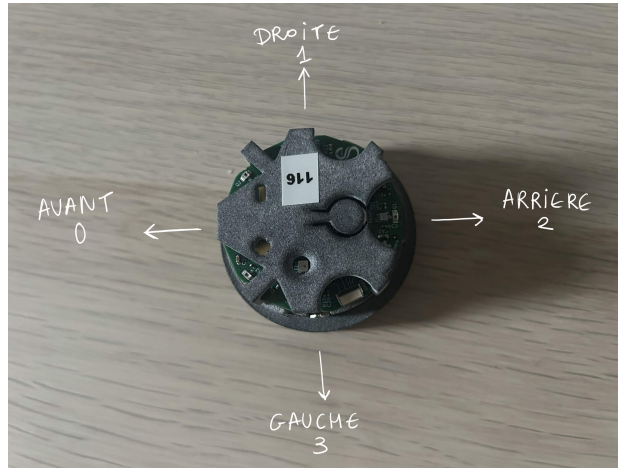


FIGURE 1 – Présentation du robot utilisé dans le projet et convention adoptée pour les faces de communication : avant (0), droite (1), arrière (2) et gauche (3).

Cette convention est importante, car une grande partie de la logique du programme dépend de la face sur laquelle un message est reçu. Par exemple, certains comportements sont déclenchés lorsqu'un signal est détecté sur la face avant, alors que d'autres correspondent à des connexions latérales ou arrière. L'orientation commune des robots permet donc de donner le même sens à ces faces pour tout l'essaim et de rendre les interactions locales cohérentes.

Le robot possède également des moteurs lui permettant d'avancer, de reculer et de tourner, ainsi qu'un système de LED utilisé pour visualiser son état. Dans le code, certaines couleurs servent d'ailleurs à représenter le mode courant du robot, par exemple pendant l'exploration, la connexion ou l'état connecté. Cependant, la convention pour utilisée les LEDs et les capteurs-émetteurs infrarouges est différente, il existe une LEDs en plus qui se trouve sur le haut du robot, les valeurs à mettre sont différentes, la convention est la suivante :

- face 0 : haut ;
- face 1 : avant ;
- face 2 : droite ;
- face 3 : arrière ;
- face 4 : gauche ;

Enfin, une armure a été ajoutée autour du robot afin d'améliorer le positionnement relatif et de mieux contraindre les échanges locaux. Cette armure aide à limiter certaines ambiguïtés physiques, même si elle ne supprime pas toutes les difficultés liées à l'alignement et à la robustesse du système.

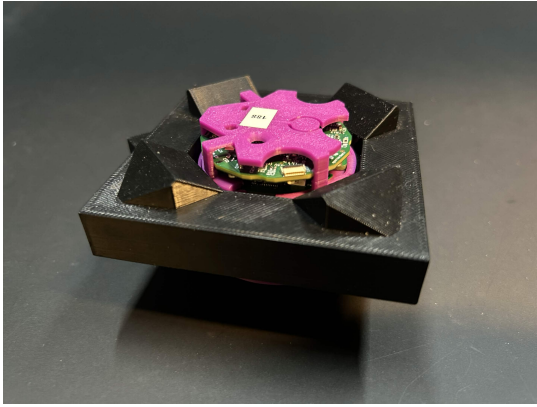


FIGURE 2 – Robot Pogobot avec armure vue de face

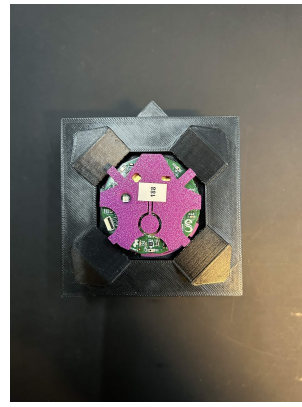


FIGURE 3 – Robot Pogobot avec armure vue de dessus

5. Organisation générale du programme

A l'exécution de l'algorithme, les robots appellent `pogobot_init()` afin d'initialiser les composants du robot et une fonction `app_init()` permettant d'initialiser les variables que l'on utilisera plus tard.

Le programme repose sur une machine à états. Chaque robot se trouve à chaque instant dans un mode particulier, associé à une fonction précise. Dans le code, ces modes sont définis dans l'énumération `mobile_mode_t`. On y retrouve ainsi les états :

- `MODE_NORMAL` ;
- `MODE_ROOT` ;
- `MODE_ALIGN` ;
- `MODE_CONNECTION_ROW` et `MODE_CONNECTION_COL` ;
- `MODE_CHECK_CONNECTION_ROW` et `MODE_CHECK_CONNECTION_COL` ;
- `MODE_CONNECTED` ;
- `MODE_ALIGN_TO_SEEK` et `MODE_SEEK_ROOT`.
- `MODE_FLAG` et `MODE_FLAG_LIGHT`

La fonction `update_mode()` exécute le comportement associé au mode courant. Cette organisation permet de découper l'assemblage en plusieurs étapes progressives : un robot peut d'abord explorer, ensuite détecter une opportunité de connexion, s'aligner, tenter une connexion, la vérifier, devenir connecté, puis participer à l'extension de la structure. Cette fonction est exécutée dans le main à une fréquence définie par une constante `FQCY`, fixée à 60Hz dans nos expériences.

D'autres constantes sont définies dans notre programme, que ce soit `IDENTIFIER` permettant de définir le robot racine ou différentes valeur de `TTL` utilisé dans nos états pour revenir à l'état initial après qu'on considère après avoir pris trop de temps dans le même état sans avoir obtenu de résultats convaincant.

Lors d'envoi de messages, à travers l'infrarouge des robots Pogobot, on utilise une structure `grid_msg_t` en tant que payload du message, contenant ainsi la position en ligne et en colonne du robot émetteur.

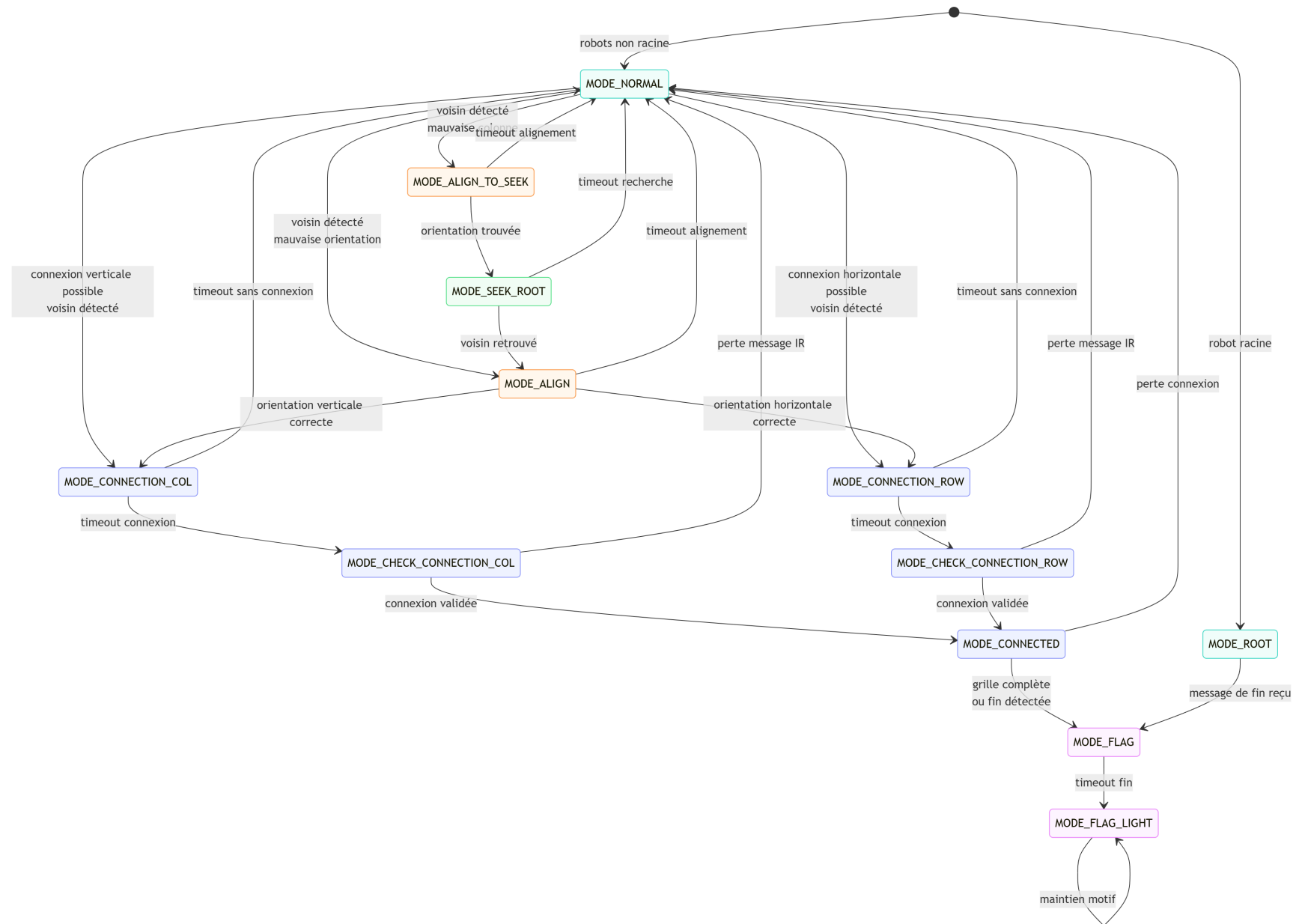


FIGURE 4 – Machine à états du comportement des Pogobots

6. Description détaillée des modes

6.1. MODE_NORMAL

Algorithm 1 MODE_NORMAL

```
1: Function UPDATE_RUN_AND_TUMBLE
2: SetColorCyan
3: if Message then
4:    $msgs \leftarrow \text{GETMESSAGES}(\text{limit} : \text{MAX\_NB})$ 
5:   for each  $m \in msgs$  do
6:     if  $m.\text{send\_ir} = 2 \wedge p.\text{col} = 0$  then
7:        $mode \leftarrow (m.\text{recv\_ir} = 0)?\text{CONN\_COL} : \text{ALIGN}$ 
8:       RESETTIMERS; break
9:     else if  $m.\text{send\_ir} = 1 \wedge p.\text{row} = 0$  then
10:       $mode \leftarrow (m.\text{recv\_ir} = 3)?\text{CONN\_ROW} : \text{ALIGN}$ 
11:      RESETTIMERS; break
12:     else if  $m.\text{send\_ir} = 2$  then
13:        $mode \leftarrow \text{ALIGN\_TO\_SEEK}$ ; break
14:     end if
15:   end for
16: end if
17:  $t_{now} \leftarrow \text{GETELAPSED}(\text{rt\_timer})$ 
18: if  $rt\_phase = \text{RUN}$  then
19:   DRIVE( $rt\_backward ? \text{REVERSE} : \text{FORWARD}$ )
20:   if  $t_{now} \geq t_{duration}$  then
21:     SWITCHPHASE( $\text{TUMBLE}, \text{rand}(\text{T\_min}, \text{T\_max}), \text{rand\_dir}()$ )
22:   end if
23: else ▷ Phase TUMBLE
24:   TURN( $rt\_dir > 0.33 ? \text{LEFT} : \text{RIGHT}$ )
25:   if  $t_{now} \geq t_{duration}$  then
26:     SWITCHPHASE( $\text{RUN}, \text{rand}(\text{R\_min}, \text{R\_max}), \text{rand\_bool}()$ )
27:   end if
28: end if
```

Le mode MODE_NORMAL correspond à la phase d'exploration libre. Il est implémenté par la fonction `update_run_tumble()`. Dans ce mode, le robot alterne entre deux sous-phases :

- une phase *RUN*, pendant laquelle il avance ou recule pendant une durée aléatoire ;
- une phase *TUMBLE*, pendant laquelle il tourne sur lui-même pendant une durée aléatoire.

Ce comportement constitue la base du projet. Il permet au robot de se déplacer sans planification globale et de rencontrer d'autres robots. Le `run_and_tumble` est un pattern qu'on retrouve dans la nature exhibé par des bactéries tel que l'*Escherichia coli* (E. Coli) [4], qui est étudié dans le domaine de la biologie et est dans la littérature de la *swarm robotics* étudié aussi [5], on retranscrit ce comportement dans notre état MODE_NORMAL.

6.2. MODE_ROOT

Algorithm 2 MODE_ROOT

```
1: Function UPDATEROOT
2: SetColorGreen
3: if Message then
4:    $msgs \leftarrow \text{GETMSGs}(\text{limit} : 3)$ 
5:   for all  $m \in msgs$  do
6:     if  $m.\text{send\_ir} \in \{0, 3\}$  then
7:        $mode \leftarrow \text{MODE\_FLAG}$ 
8:        $\text{RESETIMER}(\text{con\_timer})$ 
9:       break
10:    end if
11:  end for
12: end if
13:  $\text{SENDMSGUNISPE}((1,2), \text{PAYLOAD})$  ▷ Information de position
14:  $\text{SETLED COLOR}((2,3), \text{RED})$  ▷ On incrémente l'infrarouge par 1 pour trouver la LED correspondante
15: STOPMOTORS
```

Le mode `MODE_ROOT` correspond au robot racine. Dans le code, le robot d'identifiant 0 est initialisé directement dans ce mode et reçoit la position (0,0). Il devient donc l'origine de la structure à former.

La racine n'ayant que l'unique rôle de servir de base à la grille, on ne lui ajoute aucune autre méthode à part l'envoi de message, contenant un `grid_msg_t` comme mentionné précédemment, la réception des messages provenant de sa face arrière ou droite signifie que la grille a fini de se former et on passe à la prochaine étape de formation de motifs lumineux.

6.3. MODE_ALIGN

Algorithm 3 MODE_ALIGN

```
1: Function UPDATEALIGN
2: SetColorPurple
3: if Message then
4:    $msgs \leftarrow \text{GETMSGs}(\text{limit} : 3)$ 
5:   for all  $m \in msgs$  do
6:     if  $m.\text{send\_ir} = 2 \wedge p.\text{col} = 0 \wedge m.\text{recv\_ir} = 0$  then
7:        $mode \leftarrow \text{MODE\_CONNECTION\_COL}$ ; break
8:     else if  $m.\text{send\_ir} = 1 \wedge p.\text{row} = 0 \wedge m.\text{recv\_ir} = 3$  then
9:        $mode \leftarrow \text{MODE\_CONNECTION\_ROW}$ ; break
10:    end if
11:  end for
12: end if
13: STOPMOTORS ▷ Le robot s'arrête pour s'aligner
14: if  $\text{ELAPSED}(\text{align\_timer}) \geq \text{ALIGN\_TTL}$  then
15:    $mode \leftarrow \text{MODE\_NORMAL}$ ; return ▷ Retour si on ne s'aligne pas après un temps arbitraire défini
16: end if
```

Les modes `MODE_ALIGN` correspondent à des phases d'alignement avant une tentative de connexion. Lorsqu'un robot détecte un signal, il ne se connecte pas immédiatement, : il commence par s'orienter afin d'améliorer sa position relative, ainsi, il s'oriente à ce qu'il soit face à l'arrière du robot émetteur si c'est une connexion en colonne ou il s'oriente de façon à être parallèle pour une connexion en ligne. Dans le cas où le robot est déjà dans la bonne orientation, la machine ne passe pas par l'état `MODE_ALIGN` et passe directement dans le mode suivant.

6.4. MODE_CONNECTION_ROW et MODE_CONNECTION_COL

Algorithm 4 MODE_CONNECTION_X ($X \in \{\text{COL}, \text{ROW}\}$)

```

1: Function UPDATECONNECTION( $X$ )
2: SetColorWhite
3: if Message then
4:    $msgs \leftarrow \text{GETMSGs}(\text{limit} : 3)$ 
5:   for all  $m \in msgs$  do
6:     Payload  $p \leftarrow \text{INITPAYLOAD}$ 
7:      $col \leftarrow (X = \text{COL})?p.col+1 : p.col$ 
8:      $row \leftarrow (X = \text{ROW})?p.row+1 : p.row$ 
9:      $t_{con} \leftarrow \text{ELAPSED}(\text{con\_timer})$ 
10:     $t_{rec} \leftarrow \text{ELAPSED}(\text{rec\_input\_timer})$ 
11:   end for
12:   if  $t_{rec} > \text{REC\_TTL}$  then
13:      $mode \leftarrow \text{MODE\_NORMAL}$ ; return ▷ Perte de signal, on abandonne
14:   end if
15: end if
16: if  $t_{con} \leq \text{CON\_TTL}$  then
17:   MOTORSFORWARD
18: else ▷ On a pas perdu le signal après un temps défini, on estime qu'on s'est connecté mais on vérifie d'abord
19:    $mode \leftarrow (X = \text{COL})?\text{CHK\_COL} : \text{CHK\_ROW}$ 
20:   RESETTIMER(check_timer)
21: end if

```

Une fois aligné, le robot passe en mode de tentative de connexion. Dans le code, on distingue :

- MODE_CONNECTION_ROW, pour se connecter sur une ligne ;
- MODE_CONNECTION_COL, pour se connecter sur une colonne.

A noté qu'on ne présente qu'un pseudo-code rassemblant les 2 états mais l'algorithme sépare bien ces bouts de code en 2 états distincts, les seules différences étant les faces par lesquels sont projeter les infrarouge sur les if.

Dans le cas d'une connexion en ligne, le robot récupère la position d'un voisin et prolonge une ligne existante. Dans le cas d'une connexion en colonne, il récupère une position verticale cohérente. En différenciant ces 2 états, il permet une plus grosse robustesse en évitant des cas où des robots réinitialisent leur compteur à la réception d'un mauvais message. De plus c'est dans cet état que le robot récupère l'id du robot sur lequel il tente de se connecter et se l'attribue, si il n'arrive pas à se connecter ça ne pose pas de problème car dès qu'il passera en MODE_CONNECTION il va forcément écraser l'ancienne donnée.

6.5. MODE_CHECK_CONNECTION_ROW et MODE_CHECK_CONNECTION_COL

Algorithm 5 MODE_CHECK_CONNECTION_X ($X \in \{\text{COL}, \text{ROW}\}$)

```

1: Function UPDATECONNECTION( $X$ )
2: SetColorYellow
3: if Message then
4:    $msgs \leftarrow \text{GETMSGs}(\text{limit} : 3)$ 
5:   for all  $m \in msgs$  do
6:     if MSGFROMIR( $(X = \text{COL})?0 : 3$ ) then
7:       RESETTIMER( $\text{rec\_timer}$ )
8:     end if
9:   end for
10:  if ELAPSED( $\text{rec\_timer}$ ) > REC_TTL then
11:     $\text{mode} \leftarrow \text{MODE\_NORMAL}$ ; return ▷ Perte de signal
12:  end if
13: end if
14: if ELAPSED( $\text{check\_timer}$ ) ≤ CHK_TTL then
15:  if ELAPSED( $\text{phase\_timer}$ ) < PHASE_TTL then
16:     $(\text{phase} = \text{FW}) ? \text{MOTORSLOWFORWARD} : \text{MOTORSLOWRIGHT}$ 
17:  else
18:     $\text{phase} \leftarrow (\text{phase} = \text{FW}) ? \text{TURN} : \text{FW}$  ▷ Alternance de phase
19:    RESETTIMER( $\text{phase\_timer}$ )
20:  end if
21: else
22:   $\text{mode} \leftarrow \text{MODE\_CONNECTED}$  ▷ Vérification terminée
23: end if

```

Après la tentative de connexion, le robot ne valide pas immédiatement son intégration. Il passe d'abord dans une phase de vérification. Pendant cette phase, il continue à écouter les messages reçus et réalise de petits ajustements de mouvement. Si la connexion semble suffisamment robuste, elle est validée ; sinon, le robot retourne au mode normal.

C'est une étape importante, il peut y avoir des confusions où le robot pense être dans une position dit connectée mais est en biais par rapport au robot devant lui, ou, à côté, ainsi, avec ces mouvements constants, le robot va soit sorti du champ de réception des messages soit on confirme qu'il est bien connecté ou bien il s'est connecté grâce à cet état.

6.6. MODE_ALIGN_TO_SEEK et MODE_SEEK_ROOT

Algorithm 6 MODE_ALIGN_TO_SEEK

```

1: Function UPDATEALIGNTOSEEK
2: SetColorPurple
3: if Message then
4:    $msgs \leftarrow \text{GETMSGs}(\text{limit} : 3)$ 
5:   for all  $m \in msgs$  do
6:     if MSGFROMIR( $\text{receiver} : 1$ ) then
7:        $\text{mode} \leftarrow \text{MODE\_SEEK\_ROOT}$ ; RESETTIMER( $\text{seek\_timer}, \text{max\_timer}$ ); CLEARMSGQUEUE
8:     end if
9:   end for
10: end if
11: if ELAPSED( $\text{a\_timer}$ ) ≥ A_TTL then
12:   $\text{mode} \leftarrow \text{MODE\_NORMAL}$ ; return ▷ Trop de temps
13: end if
14: MOTORSLOWRIGHT

```

Algorithm 7 MODE_SEEK_ROOT

```
1: Function UPDATESEEKROOT
2: SetColorLightPink
3: if Message then
4:    $msgs \leftarrow \text{GETMSGs}(\text{limit} : 3)$ 
5:   for all  $m \in msgs$  do
6:     if  $(m.col = 0 \wedge m.sender = 2) \vee m.sender = 1$  then  $\triangleright$  Colonne de la racine ou double condition achevée
7:        $mode \leftarrow \text{MODE\_ALIGN}$ 
8:       RESETTIMER(a_timer)
9:     else if  $m.sender = 2$  then  $\triangleright$  On continue de seek on est en train de remonter la ligne
10:      RESETTIMER(seek_timer)
11:     end if
12:     CLEARMSGQUEUE
13:   end for
14: end if
15: if ELAPSED(seek_timer) > SEEK_TTL  $\vee$  ELAPSED(max_timer) > MAX_TTL then
16:    $mode \leftarrow \text{MODE\_NORMAL}$ 
17: else
18:   MOTORSLOWFORWARD
19: end if
```

Une étape plus avancée du projet a consisté à introduire des modes de recherche. Le robot utilise des informations qui lui sont parvenus grâce aux infrarouge pour se diriger vers des zones plus pertinentes, ainsi, si tout une ligne est complète et il reçoit une information à la droite de la ligne, de par les conditions émises, il faut qu'il se connecte sur la colonne de la racine, il va donc remonter toute la ligne pour trouver la colonne de la racine `MODE_ALIGN_TO_SEEK` lui permettra de s'aligner perpendiculairement à la ligne et `MODE_SEEK_ROOT` permet d'explorer la ligne jusqu'à trouver la ligne.

`MODE_ALIGN_TO_SEEK` et `MODE_SEEK_ROOT` possèdent une autre importance dans notre algorithme, dans le cas ou un robot voudrait se connecter à une position qui n'est ni la ligne de la racine ni la colonne de la racine, il doit remplir une double condition, qui sont représenté par ces 2 modes, `MODE_SEEK_ROOT` cherche soit la colonne de la racine soit cherche un robot qui envoie un infrarouge sur sa face droite, il sait alors qu'il peut se connecter sur une colonne qui n'est pas celle de la racine.

6.7. MODE_CONNECTED

Algorithm 8 MODE_CONNECTED

```
1: Function UPDATECONNECTED
2: SetColorGreen
3: if Message then
4:    $msgs \leftarrow \text{GETMSGs}(\text{limit} : 3)$ 
5:   for all  $m \in msgs$  do
6:     if  $m.\text{sender} = 0 \vee m.\text{sender} = 3$  then
7:        $mode \leftarrow \text{MODE\_FLAG}$ ; RESETTIMER( $finish\_timer$ ); MOTORSTOP; break
8:     end if
9:     if  $m.\text{receiver} = 3 \vee (pos.col = 0 \wedge m.\text{receiver} = 0)$  then
10:      RESETTIMER( $rec\_input\_timer$ ); CLEARMSGQUEUE
11:    end if
12:  end for
13:  if ELAPSED( $rec\_input\_timer$ ) > REC_INPUT_TTL then
14:     $mode \leftarrow \text{MODE\_NORMAL}$ ; return
15:  end if
16: end if

17: if ELAPSED( $check\_phase\_timer$ ) < CHECK_PHASE_TTL then
18:   ( $check\_phase = \text{FW}$ )? MOTORSSLOWFORWARD : MOTORSTOP
19: else
20:    $check\_phase \leftarrow (check\_phase = \text{FW})? \text{TURN} : \text{FW}$ 
21:   RESETTIMER( $check\_phase\_timer$ )
22: end if
23: SENDMSGUNISPE((1,2), PAYLOAD) ▷ Information de position
24: SETLEDCOLOR((2,3),RED)
```

Le mode MODE_CONNECTED correspond à un robot qui s'est intégré dans la structure. Dans ce mode, le robot observe en continu s'il est connecté à son environnement et repasse en MODE_NORMAL quand la vérification aura échoué. Ce mode fonctionne comme le MODE_ROOT, le robot transmettant sur sa face arrière et droite un infrarouge contenant l'information de sa position dans le grid, sous réserve qu'une connexion sur cette face ne risque pas de dépasser la taille maximale de la grid.

De plus, dans une optique de garantir une certaine robustesse et d'économie d'énergie, dans ce mode les robots alterneront entre un stop et un mouvement tout droit, dans la plupart des cas des robots se considérant comme logiquement connectés mais n'étant pas physiquement connectés finissent par se connecter avec ce système ou alors se déconnecter complètement et empêcher la formation d'une grille avec une forme anormale.

6.8. MODE_FLAG et MODE_FLAG_LIGHT

Algorithm 9 MODE_FLAG

```
1: Function UPDATEFLAG
2: SETCOLOR(Yellow)
3: SENDMSGUNISPE(0,3) ▷ Message sans contenu pour propager le mode
4: SETLEDCOLOR((1,4),RED)
5: if ELAPSED( $finish\_timer$ ) > FINISH_TTL then
6:    $mode \leftarrow \text{MODE\_FLAG\_LIGHT}$ 
7:    $flag\_light\_initialized \leftarrow \text{false}$ 
8: end if
```

Ces deux états représentent les états finaux de notre algorithme, le point culminant de notre travail. Le dernier robot se connectant à la grille passe en `MODE_FLAG` et propage une information sur la face avant et gauche, les robots en `MODE_CONNECTED` reçoivent ainsi un infrarouge sur leur face droite et arrière et passent eux aussi en `MODE_FLAG` et propagent à leur tour l'information, enfin après une certaine durée arbitraire, on considère que tous les robots sont passés en `MODE_FLAG` alors ils passent en `MODE_FLAG_LIGHT`.

Le `MODE_FLAG_LIGHT` représente l'algorithme utilisé dans les travaux A. Loi, S. Houdaibi, N. Bredeche [1] et permet au robot dans une formation en grille de s'accorder pour afficher un motif lumineux avec chacun des robots représentant un pixel, on comptait deux motifs possible au moment où l'on a fait nos expériences : la cocarde tricolore et le drapeau de l'Italie.

7. Difficultés rencontrées

L'une des principales difficultés du projet concerne la robustesse du système dans des conditions physiques réelles. Sur le papier, la logique des messages et des connexions peut sembler relativement simple : un robot reçoit une information, s'aligne, se connecte, puis transmet à son tour sa position. En pratique, cette chaîne est beaucoup plus délicate à garantir.

Le traitement des messages infrarouges constitue une première difficulté importante. Les robots doivent recevoir les bons messages, provenant du bon voisin, sur la bonne face, et dans un contexte cohérent avec leur position attendue. Or, il se peut que plusieurs phénomènes empêchent une bonne lecture des signaux, .

Une autre difficulté vient des contraintes physiques elles-mêmes. Les robots ne se déplacent pas dans un environnement idéal : leurs rotations et leurs translations peuvent être imprécises, ce qui perturbe l'alignement et rend certaines connexions fragiles. Ou encore, des cas où les valeurs en mémoire des directions des moteurs causeraient des mouvements totalement à l'opposée de ce que l'on voulait, une ligne droite devenant une rotation et inversement, ce qu'il fallait traiter au cas par cas pour chacun des robots.

Pour pallier une partie de ces problèmes, des armures ont été fournies pour les robots. Elles rappellent le rôle de l'exosquelette carré utilisé dans les expériences réelles décrites dans ANTS 2026, qui bloque la communication diagonale entre les robots [1]. Ces dispositifs améliorent la communication locale et le positionnement, mais ne suppriment pas tous les cas difficiles.

En effet, même avec ces armures, certains cas restent compliqués à traiter. Il existe encore des situations où :

- l'alignement n'est pas suffisamment précis ;
- la structure se forme de manière anormale ;
- un robot hésite entre plusieurs comportements possibles.

La plupart de ces cas proviennent des interactions entre les robots non-connectés, en effet, bien que les robots dans la grille envoient en continu des informations, les robots en dehors de la grille ne communiquent pas entre eux, ainsi on retrouve plusieurs fois des cas, où un robot tente de se connecter mais est bloqué dans son chemin par un autre robot, ainsi, après être passé dans différents états il pense être connecté car son mouvement ne l'a pas fait perdre le signal alors qu'un robot a servi de mur, la propagation du signal de ce robot anormal va ainsi créer une structure inattendue.

Un autre cas de formation anormale est dû aux socles permettant aux robots de se connecter en ligne dans la racine, le socle permet la connection de 2 robots, il faut donc en ajouter plus dans une ligne, cependant il existe une fente entre ces 2 socles dans lequel la pointe de l'armure des robots peuvent se coincer et à cette distance ils reçoivent encore le signal et se pensent connectés.

8. Conclusion

Ce travail a principalement consisté à comprendre les différentes difficultés et contraintes qui existaient dans le domaine de la *swarm robotics*. En comprenant ces difficultés, on a pu en conséquence concevoir un algorithme en prenant en compte l'importance de la robustesse, et

L'évolution du projet apparaît comme une progression cohérente. On part d'abord du *run and tumble*, qui fournit le comportement d'exploration. On introduit ensuite la racine, puis l'alignement et la première connexion. Cette logique est ensuite raffinée avec la séparation entre connexion, vérification et état connecté. Le projet progresse ensuite vers une distinction entre ligne et colonne, puis vers des comportements plus avancés comme *seek*, qui permettent de viser la formation d'une grille complète, finalement on intègre l'algorithme de formation de motifs lumineux.

Bien qu'à ce jour notre algorithme est fonctionnel, permettant de créer une grille avant d'y afficher des motifs aux travers des LEDs, il présente cependant plusieurs chemins d'améliorations :

- Ajouter une communication parmis les robots tentant de se connecter afin qu'ils ne bloquent pas la formation de la grille ;
- Modifier le comportement `MODE_NORMAL` afin de réduire le temps avant qu'un robot détecte un signal (ajout d'une mémoire ou on permet aux robots de communiquer dans des faces hors grille afin d'aiguiller les robots) ;
- Trouver des valeurs optimales de vitesse et de temps pour remplacer nos valeurs choisies arbitrairement afin de faciliter l'alignement et les mouvements en généraux.

Enfin, notre algorithme ne remplit pas tous les objectifs qui nous a été demandé en début de projet, il permet la constitution d'une grille mais le robot racine doit être désigné au préalable et on a besoin de socles permettant aux robots en ligne de s'enfoncer pour empêcher la possibilité de se faire déconnecter ou pour empêcher une déconnexion lié aux mouvements continus dans `MODE_CONNECTED`.

Références

- [1] A. Loi, S. Houdaibi, N. Bredeche. *A Modular Neural Architecture for Distributed Visual Pattern Formation in Robot Swarms*. ANTS 2026.
- [2] A. Loi, N. Bredeche. *Evolving Neural Controllers for Adaptive Visual Pattern Formation by a Swarm of Robots*. GECCO 2025.
- [3] A. Loi, N. Bredeche et al. *Pogobot : an Open-Source, Low-Cost Robot for Swarm Robotics and Programmable Active Matter*. arXiv :2504.08686, 2025.
- [4] H. C. Berg, L. Turner. *Chemotaxis of bacteria in glass capillary arrays. Escherichia coli, motility, microchannel plate, and light scattering*. Biophys J 58 : 919–930, 1990.
- [5] A. Shklarsh et al. *Smart Swarms of Bacteria-Inspired Agents with Performance Adaptable Interactions*. PLoS Computational Biology, 7(9), e1002177, 2011.
- [6] M. Brambilla, E. Ferrante, M. Birattari, M. Dorigo. *Swarm robotics : a review*. Swarm Intelligence, 2013.
- [7] M. Rubenstein, A. Cornejo, R. Nagpal. *Programmable self-assembly in a thousand-robot swarm*. Science, 2014.
- [8] D. St-Onge, et al. *Decentralized progressive shape formation with robot swarms*. Autonomous Robots, 43(8), 2137–2153, 2019.

- [9] M. Bonani et al. *The marXbot, a Miniature Mobile Robot Enhancing Micro-Engineering Competences and Research Flexibility in Swarm Robotics*. IEEE International Conference on Robotics and Automation (ICRA), 2010.

A. Cahier des charges

A.1. Contexte du projet

Ce projet s'inscrit dans le domaine de la robotique en essaim. L'objectif est de programmer un ensemble de robots de manière à ce qu'ils puissent, sans contrôle centralisé, se regrouper et s'organiser dans l'espace pour former une structure régulière. Dans un premier temps, la structure visée est une grille de taille 3×3 . Dans un second temps, cette grille sert de support à l'affichage de motifs lumineux, chaque robot jouant le rôle d'un pixel.

L'intérêt de cette approche est qu'elle permet d'obtenir un comportement collectif à partir d'interactions purement locales. Chaque robot ne dispose que d'informations limitées, transmises par ses voisins proches, mais l'ensemble du groupe doit malgré tout parvenir à produire une organisation globale cohérente. Cette logique est au coeur des travaux récents sur la formation de motifs visuels distribués dans des essaims robotiques, où les robots s'assemblent en grille puis affichent collectivement une image ou un drapeau lumineux.

A.2. Motivation

Le projet répond à deux enjeux principaux.

D'une part, il s'agit d'étudier comment un groupe de robots simples peut produire une organisation spatiale ordonnée sans chef d'orchestre ni positionnement global. La capacité à former une grille régulière constitue un cas d'étude intéressant, car elle demande aux robots de se positionner correctement les uns par rapport aux autres à partir d'informations locales seulement.

D'autre part, cette grille n'est pas une fin en soi : elle permet ensuite de transformer l'essaim en surface d'affichage distribuée. Une fois organisés, les robots peuvent afficher un motif lumineux collectif, par exemple une forme simple, un symbole ou un drapeau. Dans cette logique, chaque robot correspond à un pixel de l'image finale.

A.3. Pourquoi une grille et une orientation commune ?

Le choix d'une grille répond à plusieurs besoins fonctionnels. Une grille fournit d'abord une structure régulière, simple à interpréter et à évaluer. Elle permet d'associer à chaque robot une position relative claire dans l'ensemble. Cette organisation facilite ensuite l'affichage de motifs, puisque chaque robot peut être vu comme une case ou un pixel d'une image collective.

Le projet suppose également que tous les robots partagent la même orientation. Ce choix simplifie fortement la construction du comportement collectif. Si tous les robots sont orientés dans le même sens, alors les directions « devant », « derrière », « gauche » et « droite » ont la même signification pour tous. Cela rend les échanges locaux plus cohérents et permet d'identifier plus facilement les voisins utiles à la formation des lignes et des colonnes.

Ainsi, le choix d'une grille combiné à une orientation commune permet :

- d'obtenir une structure finale régulière et lisible ;
- d'associer à chaque robot une position relative claire ;
- de faciliter l'affichage de motifs lumineux ;
- d'interpréter les messages reçus de manière identique ;
- de distinguer correctement les voisins latéraux et frontaux ;
- de simplifier la détection des cas particuliers comme les coins, les bords ou le centre.

A.4. Objectif général

L'objectif général du projet est de concevoir un ensemble de comportements distribués permettant à plusieurs robots :

1. de se regrouper progressivement ;
2. de s'aligner de manière ordonnée ;
3. de former une grille régulière de taille 3×3 ;
4. d'identifier leur rôle local dans cette grille ;
5. d'afficher ensuite un motif lumineux collectif.

A.5. Besoins exprimés par le « client »

Le besoin peut être formulé de manière simple de la façon suivante :

Je souhaite qu'un groupe de robots soit capable de s'organiser tout seul pour former une petite grille carrée régulière. Une fois cette grille formée, je souhaite que les robots puissent afficher ensemble un motif lumineux lisible, chaque robot représentant un point de ce motif.

Le système attendu doit donc :

- fonctionner de manière distribuée ;
- ne pas dépendre d'un contrôleur central ;
- exploiter uniquement des informations locales ;
- produire une organisation stable et reproductible ;
- permettre un affichage collectif après la phase d'organisation.

A.6. Fonction principale attendue

Former automatiquement une grille de robots de taille 3×3 , puis utiliser cette grille comme support d'affichage d'un motif lumineux collectif.

A.7. Fonctions détaillées attendues

A.7.1 Gestion des cas locaux

Le comportement doit tenir compte de plusieurs situations particulières au cours de la formation de la grille :

- **Cas de la première colonne** : les robots appartenant au bord gauche de la grille n'ont pas de voisin sur leur gauche ; leur comportement doit rester cohérent malgré cette absence.
- **Cas de la première ligne** : les robots appartenant au bord supérieur de la grille n'ont pas de voisin au-dessus ; cette situation doit être détectée correctement.
- **Cas d'un angle formé par trois robots** : lorsqu'une structure en coin ou en « L » apparaît, le robot concerné doit reconnaître qu'il participe à un angle de la future grille.
- **Cas des coins** : les robots en coin ont deux absences de voisins orthogonaux ; ils constituent des repères structurants pour la forme finale.
- **Cas du centre** : le robot central, une fois la grille formée, doit être entouré de voisins sur les quatre côtés.

A.7.2 Affichage du motif

Une fois la grille stabilisée, les robots doivent :

- déterminer quelle valeur lumineuse afficher ;
- contribuer à un motif collectif lisible ;
- maintenir cet affichage de manière stable tant que la structure reste valide.

A.8. Contraintes, périmètre et critères de réussite

Le projet doit respecter plusieurs contraintes. Le contrôle doit être entièrement distribué, sans coordinateur central. Les robots ne disposent que d'informations locales et la communication doit rester limitée aux voisins proches. L'algorithme doit rester suffisamment simple pour être embarqué sur les robots. Dans cette première version, l'objectif expérimental est limité à la formation d'une grille 3×3 .

Le système doit également être robuste : il doit tolérer les phases transitoires désordonnées, converger vers une structure stable et limiter autant que possible les ambiguïtés de placement. Le comportement attendu doit rester lisible, de sorte que les étapes de formation de la grille et les cas locaux importants puissent être interprétés facilement.

Le périmètre du premier prototype couvre la formation d'une grille carrée de petite taille, en particulier une grille 3×3 , ainsi que l'affichage d'un motif lumineux simple une fois cette structure stabilisée. En revanche, les grilles de plus grande taille, les motifs complexes et la gestion de perturbations importantes ne font pas partie des objectifs prioritaires à ce stade.

Le projet sera considéré comme réussi si les robots parviennent à former une grille 3×3 stable, si les rôles locaux (coin, bord, centre) sont cohérents avec la structure obtenue, si les cas particuliers comme la première ligne, la première colonne et les angles sont correctement gérés, et si un motif lumineux collectif peut être affiché à la fin.

Le prototype ne couvre pas nécessairement, à ce stade :

- les grilles de grande taille ;
- les motifs complexes en haute résolution ;
- les perturbations importantes ou pertes multiples de robots.

B. Manuel d'utilisation

Cette section décrit la manière d'utiliser le programme sur les robots Pogobots afin de reproduire les expériences de formation de grille et d'affichage lumineux.

B.1. Matériel nécessaire

Pour exécuter les expériences, il faut disposer :

- de plusieurs robots Pogobots ;
- des armures permettant de contraindre l'alignement ;
- d'au moins un socle pour le robot racine ;
- d'un ordinateur permettant de compiler et téléverser le programme ;
- de l'environnement logiciel fourni avec l'API Pogobot.

Pour une expérience complète de grille 3×3 , il faut idéalement disposer de neuf robots.

B.2. Compilation du programme

Le programme est écrit en langage C en utilisant l'API officielle des Pogobots. Une fois le projet récupéré, la compilation peut être réalisée à l'aide des outils fournis dans l'environnement Pogobot.

Après compilation, le fichier généré doit être téléversé sur chacun des robots utilisés pendant l'expérience.

B.3. Configuration de l'identifiant du robot

Avant de compiler le programme, il est nécessaire de modifier la constante suivante dans le fichier source :

```
#define IDENTIFIER X
```

Cette valeur détermine le rôle initial du robot :

- `IDENTIFIER 0` : robot racine ;
- `IDENTIFIER 1` : robot classique.

Lors de nos expériences :

- un unique robot possède l'identifiant 0 ;
- tous les autres robots utilisent l'identifiant 1.

Le robot d'identifiant 0 démarre directement dans le mode `MODE_ROOT`, tandis que les autres robots démarrent dans le mode `MODE_NORMAL`.

Exemple :

```
#define IDENTIFIER 0    // robot racine
#define IDENTIFIER 1    // autres robots
```

B.4. Initialisation des robots

Avant de lancer l'expérience :

- vérifier que les batteries des robots sont suffisamment chargées ;
- vérifier que les armures sont correctement fixées ;
- placer le robot racine dans son socle ;
- allumer ensuite les autres robots.

Une fois allumés, les robots commencent automatiquement l'exécution de l'algorithme.

B.5. Déroulement d'une expérience

L'expérience se déroule généralement selon les étapes suivantes :

1. les robots explorent l'environnement avec le comportement *run and tumble* ;
2. les robots détectent progressivement la racine ou des robots déjà connectés ;
3. des connexions locales se forment ;
4. la grille se construit progressivement ;
5. une fois la grille considérée comme complète, les robots passent dans les états `MODE_FLAG` puis `MODE_FLAG_LIGHT` ;
6. le motif lumineux est affiché collectivement.

Selon les conditions physiques et les collisions entre robots, plusieurs tentatives peuvent être nécessaires avant d'obtenir une grille stable.

B.6. Couleurs utilisées

Les LEDs des robots permettent de visualiser l'état courant du système :

- Cyan : exploration (`MODE_NORMAL`) ;
- Vert : robot racine ou robot connecté ;
- Violet : alignement ;
- Blanc : tentative de connexion ;
- Jaune : vérification de connexion et propagation du signal de fin ;
- Rose clair : mode `MODE_SEEK_ROOT`.

Certaines LEDs rouges sont également utilisées localement pour représenter les faces de communication infrarouge actives.

Ces couleurs permettent de suivre facilement l'évolution de la formation de la grille pendant l'expérience.

B.7. Conseils d'utilisation

Afin d'améliorer les résultats expérimentaux :

- utiliser une surface relativement plane ;
- éviter les obstacles autour de la zone d'expérience ;
- espacer légèrement les robots au départ ;
- vérifier l'orientation correcte des armures ;
- éviter les sources lumineuses infrarouges trop fortes.

Il est également conseillé de tester le comportement avec un nombre réduit de robots avant de lancer une expérience complète.

B.8. Limites connues

Certaines situations peuvent encore provoquer des comportements anormaux :

- blocage physique entre robots ;
- faux alignements ;
- connexions considérées valides alors qu'elles sont incorrectes ;
- formations incomplètes de la grille.

Ces problèmes proviennent principalement des contraintes physiques des robots et de la communication infrarouge locale.

B.9. Arrêt de l'expérience

L'expérience peut être arrêtée simplement en éteignant les robots. Lors d'une nouvelle exécution, les robots redémarrent automatiquement depuis leur mode initial.